Practical:11 **Date:22-10-2026**

Great Feature Engineering

Procedure:

Step 1: Import the required Python libraries, including Pandas, NumPy, Matplotlib and Scikit-learn.

Step 2: Load the Ames Housing training and testing datasets using Pandas.

Step 3: Inspect the dataset by checking the first few observations, missing values, data types and summary statistics.

Step 4: Separate the target variable (SalePrice) from the explanatory variables.

Step 5: Identify numerical and categorical variables in the dataset.

Step 6: Handle missing numerical values using median imputation and categorical values using mode imputation.

Step 7: Transform categorical variables into numerical form using One-Hot Encoding.

Step 8: Split the training dataset into training and validation sets in an 80:20 ratio.

Step 9: Train the Random Forest Regression model using the training dataset.

Step 10: Predict house prices using the validation dataset.

Step 11: Evaluate the model performance using RMSE, MAE and R-squared.

Step 12: Visualize the distribution of house prices, compare actual and predicted prices, and identify important features using graphs.

Step 13: Generate house price predictions for the test dataset and save the results in a CSV file.

Solution:

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
# STEP 2: LOAD THE DATASET
DATA_DIR = r"C:\Users\Kavya Shettigar\Downloads"
train_path = os.path.join(DATA_DIR, "train.csv")
test_path = os.path.join(DATA_DIR, "test.csv")
train = pd.read_csv(train_path)
test = pd.read_csv(test_path)
print("Training Dataset Shape:", train.shape)
print("Testing Dataset Shape:", test.shape)
```

```
# STEP 3: INSPECT THE DATASET
print("\nFirst Five Rows:")
print(train.head())
print("\nDataset Information:")
train.info()
print("\nMissing Values:")
print(train.isnull().sum().sort_values(ascending=False).head(20))
print("\nData Types:")
print(train.dtypes)
print("\nStatistical Summary:")
print(train.describe())
# STEP 4: DATA PREPROCESSING
X = train.drop(["SalePrice", "Id"], axis=1)
y = train["SalePrice"]
numerical_features = X.select_dtypes(
    include=["int64", "float64"]
).columns
categorical_features = X.select_dtypes(
    include=["object", "category"]
).columns
```

```
numerical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median"))
])
categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
preprocessor = ColumnTransformer(transformers=[
    ("num", numerical_pipeline, numerical_features),
    ("cat", categorical_pipeline, categorical_features)
])
X_train, X_val, y_train, y_val = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
print("\nTraining Data Shape:", X_train.shape)
print("Validation Data Shape:", X_val.shape)
```

```
# STEP 6: TRAIN RANDOM FOREST REGRESSION MODEL
model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", RandomForestRegressor(
        n_estimators=200,
        random_state=42,
        n_jobs=-1
    ))
])
model.fit(X_train, y_train)
print("\nRandom Forest Model Training Completed!")
# STEP 7: MAKE PREDICTIONS
y_pred = model.predict(X_val)
# STEP 8: EVALUATE MODEL PERFORMANCE
rmse = np.sqrt(mean_squared_error(y_val, y_pred))
mae = mean_absolute_error(y_val, y_pred)
r2 = r2_score(y_val, y_pred)
print("\nMODEL PERFORMANCE")
print("------------------------")
print("Root Mean Squared Error (RMSE):", round(rmse, 2))
```

```
print("Mean Absolute Error (MAE):", round(mae, 2))
print("R-squared (R2):", round(r2, 4))
# STEP 9: COMPARE ACTUAL AND PREDICTED PRICES
comparison = pd.DataFrame({
    "Actual Price": y_val.values,
    "Predicted Price": y_pred
})
print("\nActual vs Predicted House Prices:")
print(comparison.head(10))
# STEP 10: VISUALIZATION
# Graph 1: Distribution of House Prices
plt.figure(figsize=(8, 5))
plt.hist(y, bins=30, edgecolor="black")
plt.title("Distribution of House Prices")
plt.xlabel("Sale Price")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
# Graph 2: Actual vs Predicted House Prices
plt.figure(figsize=(8, 5))
plt.scatter(y_val, y_pred, alpha=0.6)
plt.xlabel("Actual House Prices")
```

```
plt.ylabel("Predicted House Prices")
plt.title("Actual vs Predicted House Prices")
minimum = min(y_val.min(), y_pred.min())
maximum = max(y_val.max(), y_pred.max())
plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    "r--"
)
plt.tight_layout()
plt.show()
# Graph 3: Feature Importance
rf_model = model.named_steps["regressor"]
feature_names = model.named_steps[
    "preprocessor"
].get_feature_names_out()
importance = pd.Series(
    rf_model.feature_importances_,
    index=feature_names
).sort_values(ascending=False).head(15)
plt.figure(figsize=(9, 6))
importance.sort_values().plot(kind="barh")
```

```python
plt.title("Top 15 Important Features")
plt.xlabel("Feature Importance")
plt.ylabel("Features")
plt.tight_layout()
plt.show()
# STEP 11: PREDICT PRICES FOR TEST DATASET
test_ids = test["Id"]
test_features = test.drop(["Id"], axis=1)
test_predictions = model.predict(test_features)
submission = pd.DataFrame({
    "Id": test_ids,
    "SalePrice": test_predictions
})
submission.to_csv(
    os.path.join(DATA_DIR, "Ames_Housing_Predictions.csv"),
    index=False
)
print("\nTest Dataset Predictions Saved Successfully!")
print("File: Ames_Housing_Predictions.csv")
```

```
Training Dataset Shape: (1460, 81)
Testing Dataset Shape: (1459, 80)

First Five Rows:
   Id  MSSubClass MSZoning  LotFrontage  LotArea Street Alley LotShape LandContour  ... PoolQC Fence MiscFeature MiscVal MoSold YrSold SaleType SaleCondition  SalePrice
0   1          60       RL         65.0     8450   Pave   NaN      Reg         Lvl  ...    NaN   NaN         NaN       0      2   2008       WD        Normal     208500
1   2          20       RL         80.0     9600   Pave   NaN      Reg         Lvl  ...    NaN   NaN         NaN       0      5   2007       WD        Normal     181500
2   3          60       RL         68.0    11250   Pave   NaN      IR1         Lvl  ...    NaN   NaN         NaN       0      9   2008       WD        Normal     223500
3   4          70       RL         60.0     9550   Pave   NaN      IR1         Lvl  ...    NaN   NaN         NaN       0      2   2006       WD       Abnorml     140000
4   5          60       RL         84.0    14260   Pave   NaN      IR1         Lvl  ...    NaN   NaN         NaN       0     12   2008       WD        Normal     250000

[5 rows x 81 columns]
```

```
Missing Values:
PoolQC           1453
MiscFeature      1406
Alley            1369
Fence            1179
MasVnrType        872
FireplaceQu       690
LotFrontage       259
GarageQual         81
GarageFinish       81
GarageType         81
GarageYrBlt        81
GarageCond         81
BsmtFinType2       38
BsmtExposure       38
BsmtCond           37
BsmtQual           37
BsmtFinType1       37
MasVnrArea          8
Electrical          1
Condition2          0
dtype: int64

Data Types:
Id                int64
MSSubClass        int64
MSZoning         object
LotFrontage     float64
LotArea           int64
                 ...
MoSold            int64
YrSold            int64
SaleType         object
SaleCondition    object
SalePrice         int64
Length: 81, dtype: object
```

Statistical Summary:

	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	...	ScreenPorch	PoolArea	MiscVal	MoSold	YrSold	SalePrice
count	1460.000000	1460.000000	1201.000000	1460.000000	1460.000000	1460.000000	...	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000	1460.000000
mean	730.500000	56.897260	70.049958	10516.828082	6.099315	5.575342	...	15.060959	2.758904	43.489041	6.321918	2007.815753	180921.195890
std	421.610009	42.300571	24.284752	9981.264932	1.382997	1.112799	...	55.757415	40.177307	496.123024	2.703626	1.328095	79442.502883
min	1.000000	20.000000	21.000000	1300.000000	1.000000	1.000000	...	0.000000	0.000000	0.000000	1.000000	2006.000000	34900.000000
25%	365.750000	20.000000	59.000000	7553.500000	5.000000	5.000000	...	0.000000	0.000000	0.000000	5.000000	2007.000000	129975.000000
50%	730.500000	50.000000	69.000000	9478.500000	6.000000	5.000000	...	0.000000	0.000000	0.000000	6.000000	2008.000000	163000.000000
75%	1095.250000	70.000000	80.000000	11601.500000	7.000000	6.000000	...	0.000000	0.000000	0.000000	8.000000	2009.000000	214000.000000
max	1460.000000	190.000000	313.000000	215245.000000	10.000000	9.000000	...	480.000000	738.000000	15500.000000	12.000000	2010.000000	755000.000000

[8 rows x 38 columns]

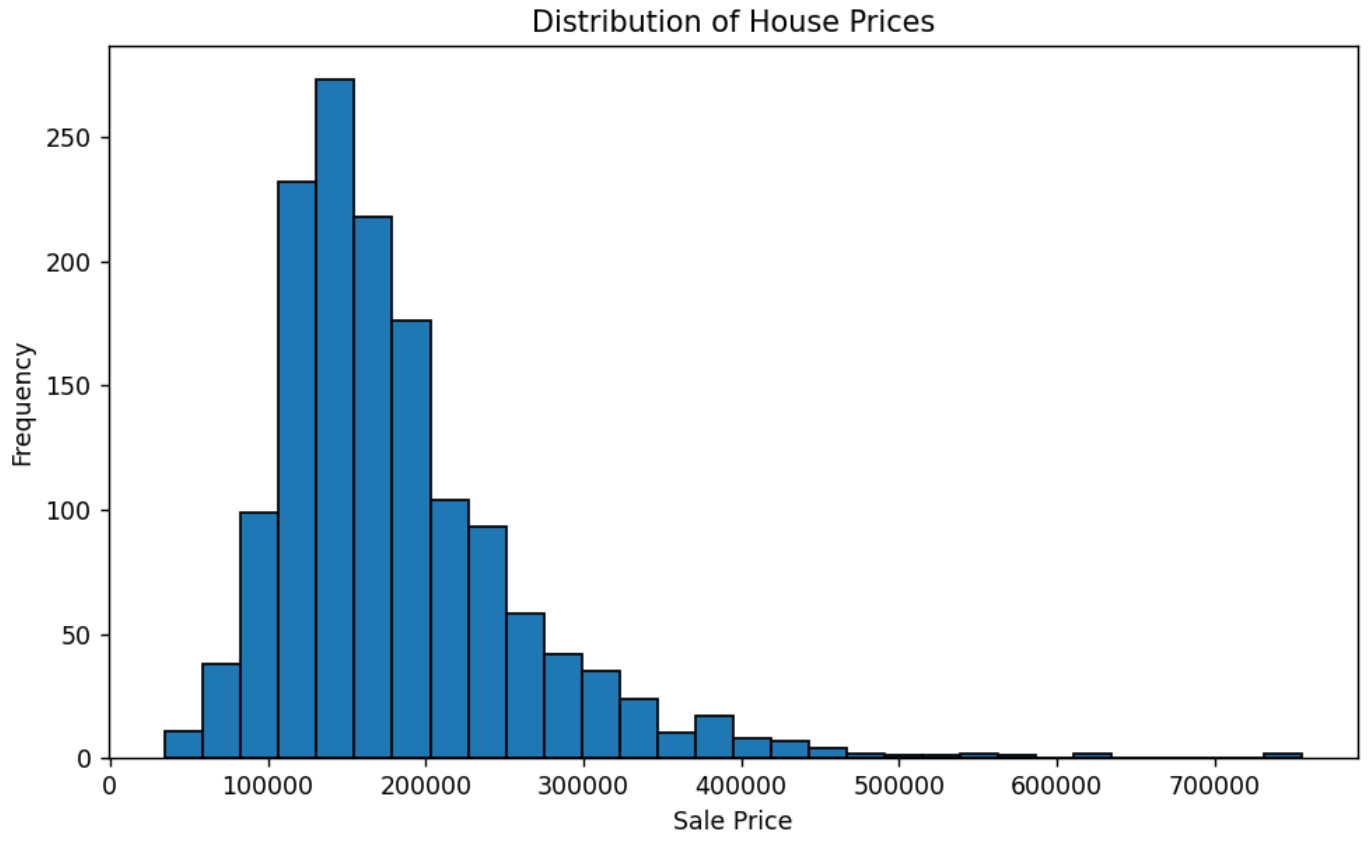
Training Data Shape: (1168, 79)
Validation Data Shape: (292, 79)

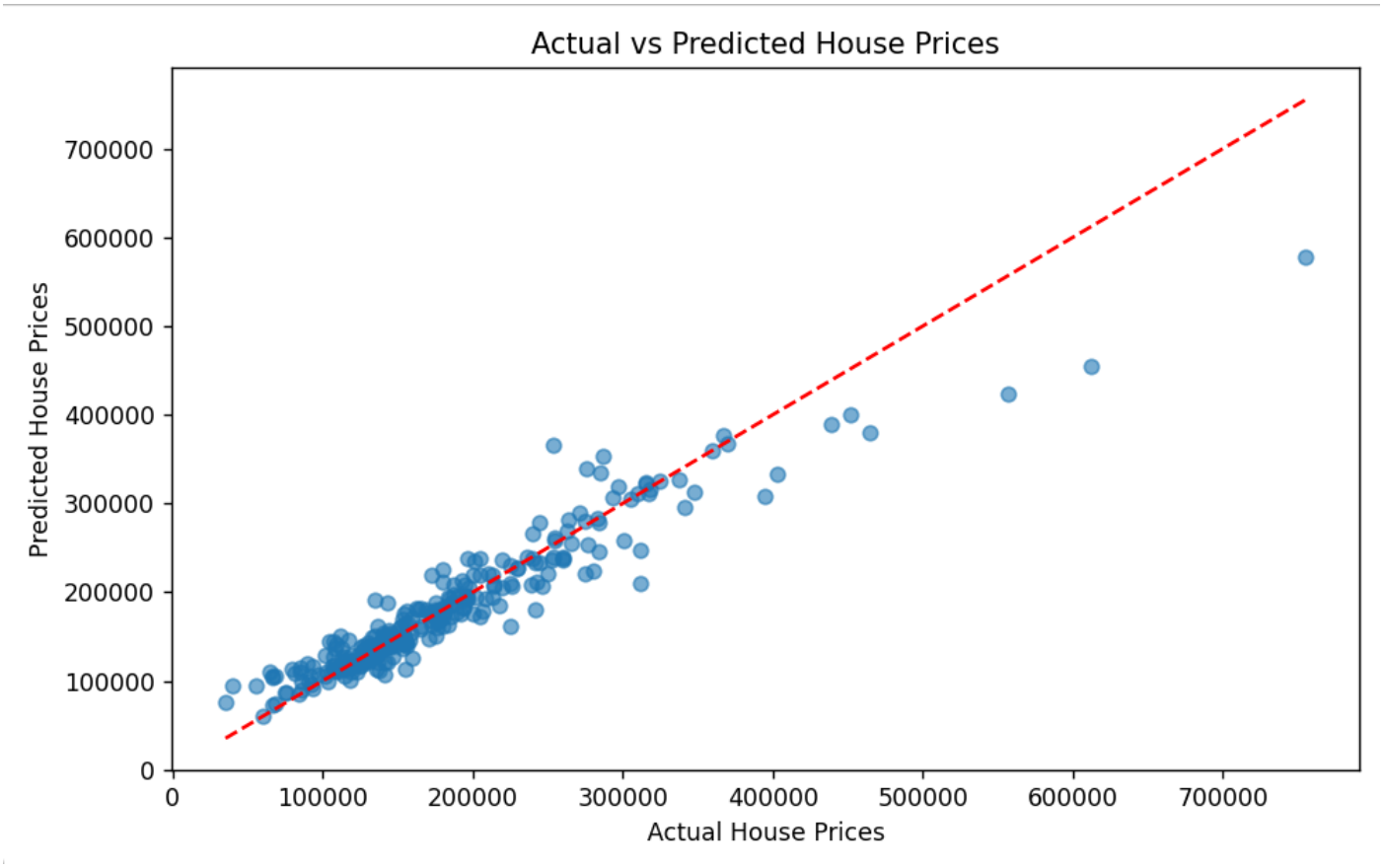
MODEL PERFORMANCE

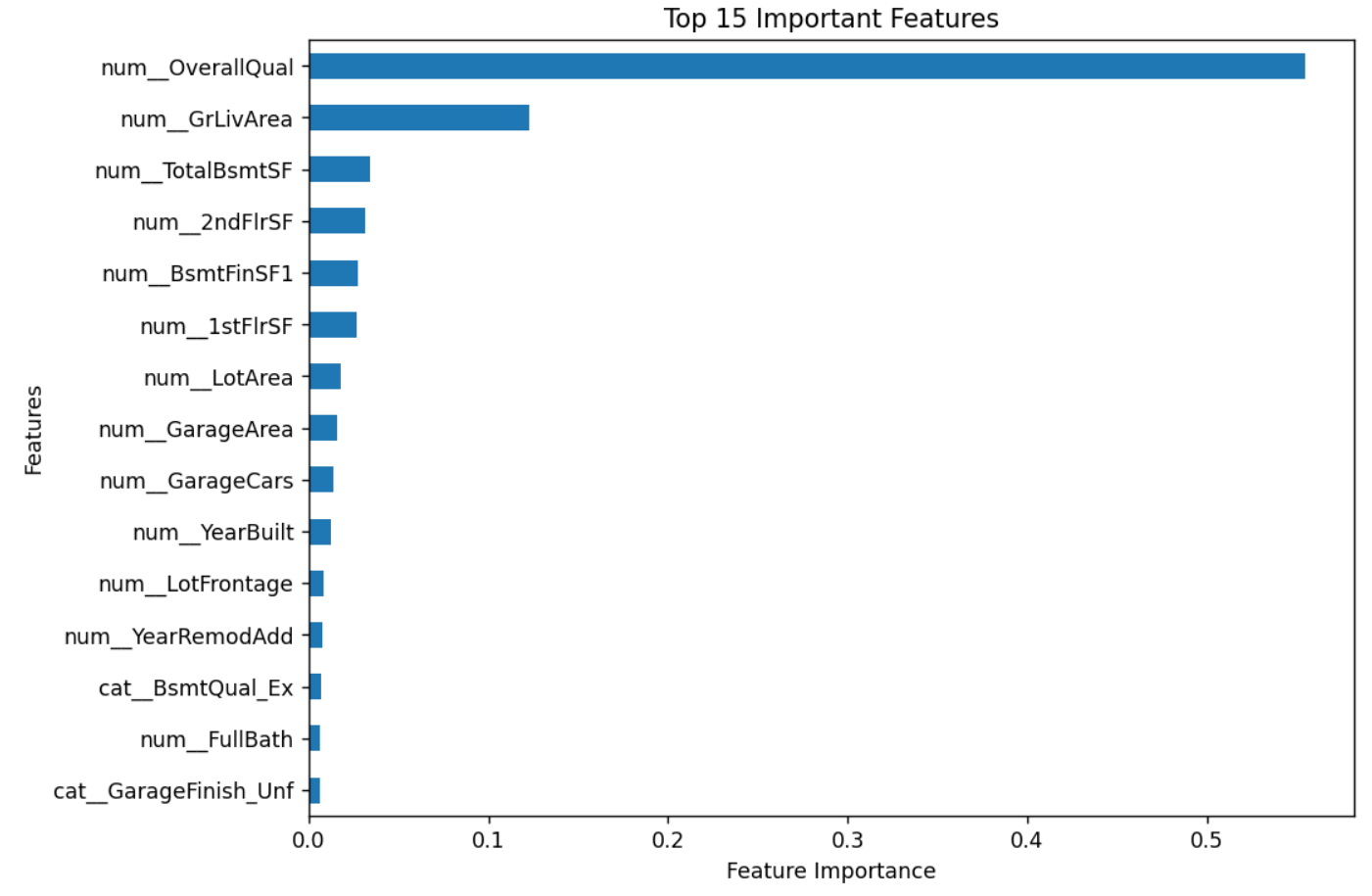
Root Mean Squared Error (RMSE): 28495.23
Mean Absolute Error (MAE): 17386.11
R-squared (R2): 0.8941

Actual vs Predicted House Prices:

	Actual Price	Predicted Price
0	154500	140679.875
1	325000	325016.005
2	115000	115839.035
3	159000	153085.370
4	315500	322853.315
5	75500	86631.750
6	311500	209628.035
7	146000	151772.250
8	84500	85337.745
9	135500	129267.185




Actual vs Predicted House Prices
Predicted House Prices
Actual House Prices
0
100000
200000
300000
400000
500000
600000
700000


Top 15 Important Features
Features
num__OverallQual
num__GrLivArea
num__TotalBsmtSF
num__2ndFlrSF
num__BsmtFinSF1
num__1stFlrSF
num__LotArea
num__GarageArea
num__GarageCars
num__YearBuilt
num__LotFrontage
num__YearRemodAdd
cat__BsmtQual_Ex
num__FullBath
cat__GarageFinish_Unf
0.0
0.1
0.2
0.3
0.4
0.5
Feature Importance